appending `L[i]` to `key`, and then hash them using `H` (remember that well designed hash functions will produce very different results for small changes in the input)

- Use double or triple hashing[7].

While the latter won't guarantee independence among the hash functions generated, it has been proven[8] that we can relax this constraint with a minimal increase in false positive rate. To keep things simple, in our implementation we use double hashing with two independent hash functions: *Murmur hashing*[9] and *Fowler-Noll-Vo*[10] hashing.

The general format of our `i-th` hash function, for `I` between `0` and `k-1`, will be:

```
h_i(key) = murmurhash(key) + i * fnv1(key) + i * i.
```

4.6 Implementation

Enough with the theory, now it's time to get once again our hands dirty! As usual, in the next sections we'll show pseudocode snippets and comment the key sections. Trivial methods will be omitted. On the book's repo on GitHub, you can also find an implementation with the full code along with unit tests.

4.6.1 Using a Bloom Filter